



WORKSHOP BRIEF — DEVELOPER ENABLEMENT

Events Board

Build brief and task specification for the Claude Code practical session

Document reference	KAS-WS-0014-01
Revision	Revision 1.0
Date of issue	17 August 2026
Subject	An internal events board built from a supplied skeleton in a timed session
Prepared for	Kiloma development team — practical session participants
Purpose	To specify what each participant is to build, the rules the implementation must satisfy, and the basis on which the results will be compared
Determination	FOR ACTION — 80-MINUTE BUILD
Classification	Confidential — internal distribution only

This document and the specification it contains are the property of Kiloma Advanced Solutions Ltd. All rights reserved. It is supplied in confidence and may not be reproduced, disclosed, distributed or used, in whole or in part, for any purpose other than the internal training exercise described herein, without the prior written consent of Kiloma Advanced Solutions Ltd. The Kiloma name and logo are trademarks of Kiloma Advanced Solutions Ltd.

1 Scope and objective

Each participant has eighty minutes and Claude Code to build an internal events board on top of a supplied skeleton.

Everything that is not the product has already been built and is not to be rebuilt: the design system, the component kit, the data layer, the session handling and the API conventions. The whole of the available time is to be spent on product behaviour — visibility rules, registration flows and the screens that expose them.

Every participant works from the same starting commit, on their own branch, against the same specification. The results are compared at the end of the session, which is why the rules in section 5 are stated as tables rather than prose: they are the standard against which each implementation is assessed.

EXPECTED OUTCOME

No participant is expected to complete every milestone. Milestones M1 to M3 are mandatory and are to be finished properly before anything else is attempted. A pull request that states honestly that M4 was not started is preferred to one that claims more than the code delivers.

2 The product

An internal events board with two halves. **Management:** authorised users create, edit and delete events, each of which opens up in one of three ways. **Discovery:** everyone else browses the events they are permitted to see and registers for the ones they want.

2.1 Access modes

Mode	Who can see it	What registering does
Open	Everyone	Confirms the registrant immediately
Approval needed	Everyone	Creates a request that a host decides on
Invite only	Hosts and invited people only	Confirms the registrant immediately

2.2 Design constraints

Two constraints shape the design and are not negotiable.

There is no separate edit screen.

Hosts edit an event on that event's own detail page, where additional controls appear for them. Everyone receives the same screen; hosts receive more of it. A second, near-identical edit screen is precisely the duplication this constraint exists to prevent.

The board must not leak.

An invite-only event does not appear on the board for a person who was not invited, and it does not appear in the API response either. Filtering it out in the browser does not satisfy this requirement.

3 Session rules

1. Work on your own branch, named <yourname>/events-board.
2. Commit as you go. The commit history forms part of the assessment.
3. Open a pull request when the work is done or when the eighty minutes elapse, whichever comes first.
4. State in the pull request what was not done. Accuracy is assessed; completeness alone is not.

4 Provided baseline

Run `npm run dev` and open `http://localhost:3000`. The start page lists everything that is provided, and `/styleguide` renders every component against real data. Both are to be read before any CSS is written.

4.1 Repository contents

Location	Contents
<code>app/styles/tokens.css</code>	Colour, spacing, type, radii and shadow tokens, light and dark
<code>app/globals.css</code>	Reset and base element styling
<code>components/ui/</code>	Button, Badge, Card, Field, Input, Select, Textarea, Modal, ConfirmDialog, Toast, Avatar, EmptyState, Spinner, SegmentedControl, PageHeader
<code>components/events/</code>	EventCard, EventGrid, DateBlock, AccessBadge, EventStatusBadge, RegistrationBadge, CapacityMeter, EventMetaLine, EventMetaDetails
<code>components/layout/</code>	Application shell, navigation and persona switcher
<code>lib/types.ts</code>	The domain model. To be read first.
<code>lib/db.ts</code>	Asynchronous in-memory store, shaped so a real database could replace it
<code>lib/seed.ts</code>	Twelve events and five people, covering every state
<code>lib/session.ts</code>	<code>getCurrentUser()</code> — the trusted identity on the server
<code>lib/api.ts</code>	<code>withErrorHandling</code> , <code>ApiError</code> , <code>jsonOk</code> , <code>readJson</code> , <code>fetchJson</code>
<code>lib/labels.ts</code>	The product's user-facing vocabulary
<code>lib/date.ts</code>	Date formatting and calendar grouping
<code>app/api/session/route.ts</code>	The worked example of the API house style

4.2 Test personas

There is no authentication. Identity is selected from the persona switcher in the top right, which sets a cookie the server reads on every request. Switching persona is the means by which visibility rules are verified.

Persona	Role	Verifies
Maya Cohen	organizer	Hosting, editing, the approval queue
Daniel Ross	organizer	An organizer viewing another organizer's event
Priya Nair	member	The plain attendee experience
Tom Alvarez	member	A participant holding a rejected request
Sara Klein	admin	Managing an event the person does not host

ENVIRONMENT NOTE

Data is held in memory and resets whenever the development server restarts. The fixtures can be restored without a restart by issuing `POST /api/dev/reset`. There is no database, and none is required.

5 Specification

The following is the specification against which each implementation is assessed. Where it is silent, participants are to exercise judgement and record the decision in the pull request.

5.1 Roles

Role	Permitted
member	Browse and register
organizer	The above, and create events, and manage the events they host
admin	Manage every event

A **host** is the event's `organizerId`, or any user listed in its `coHostIds`.

5.2 Event visibility

Event	Member, not invited	Invited member	Host	Admin
draft, any access	no	no	yes	yes
published+open	yes	—	yes	yes
published+approval	yes	—	yes	yes
published+invite	no	yes	yes	yes
cancelled	as if published		yes	yes

REQUIRED BEHAVIOUR

An event the requester is not permitted to see is to return **404**, not 403. A 403 response confirms that the event exists, which is itself a disclosure.

5.3 Permitted actions

Action	Member	Organizer, not a host	Host	Admin
Create an event	no	yes	—	yes
Edit or delete an event	no	no	yes	yes
Publish a draft	no	no	yes	yes
Approve or reject requests	no	no	yes	yes
Register	yes	yes	yes	yes

A host is not automatically an attendee, and registers on the same terms as anyone else.

5.4 Registration outcomes

Access mode	Resulting status
open	going
approval	pending, which a host resolves to going or rejected
invite, registrant invited	going

5.5 Capacity and closure

Registration is **closed** when any of the following holds: the event is a draft or is cancelled; the event has already started; or the event is full and its mode is open or invite.

An event is **full** when the number of going registrations reaches capacity. A capacity of null denotes no limit. Only going registrations count toward capacity — pending, cancelled and rejected do not.

On an approval-mode event that is full, requests may still be submitted, but a host is not permitted to approve beyond capacity.

5.6 Withdrawal

Any person whose status is going or pending may withdraw, which sets their status to cancelled. They may register again afterwards. A person whose request was rejected may not submit a further request, though a host may still approve them from the queue.

6 Milestones

M1 — The board **MANDATORY**

`/events` presents the events the current person is permitted to see. Built from `EventCard` within `EventGrid`; no replacement card is to be written. Upcoming events first, soonest first, with past events separated and de-emphasised. Each card shows the viewer's own registration status where one exists. Filtering by category and by access mode. A considered empty state, not a blank page.

Acceptance — switching from Maya to Priya removes the two invite-only events from the board and the count changes accordingly.

M2 — The event detail screen **MANDATORY**

One screen serving three audiences. It presents everything about the event — timing, location, description, hosts, capacity and attendees — and returns 404 to anyone not permitted to see it. The primary call to action reflects the viewer's actual state: register, request a place, withdraw, awaiting approval, event full, or event passed. Host-only controls appear here and only for hosts.

Acceptance — opening an invite-only event's URL directly as Tom returns 404.

M3 — Register and respond **MANDATORY**

Route handlers under `/api/events/[id]/registrations`, following the house style established in `app/api/session/route.ts`. Every rule in section 5 is enforced **on the server**; the client is to be assumed hostile. The interface updates without a full page reload and surfaces errors returned by the server.

Acceptance — calling the registration endpoint directly, as a member, for an invite-only event to which that member was not invited, fails with an appropriate status.

M4 — Create and edit in place **EXPECTED**

Hosts edit the event on its detail page; no second screen is introduced. The creation flow is the participant's own design decision — a dedicated route, a modal, or creating a draft and landing on its detail page — and is to be justified in the pull request. Deletion requires confirmation; `ConfirmDialog` is provided.

M5 — The approval queue **EXPECTED**

Hosts see pending requests on the event detail page together with each requester's message, and can approve or reject them individually. Counts and the attendee list update immediately.

Stretch objectives **OPTIONAL**

In the order they would be picked up: a calendar view on `/events` toggled with `SegmentedControl`, for which `toDayKey()` is provided; a waitlist, for which the `waitlisted` status already exists, with automatic promotion on withdrawal; host management of invitations; search across title, summary and description; a personal view of events hosted, attending and awaiting approval; optimistic updates on the registration control; and a theme toggle, for which the tokens already support `data-theme`.

7 Points of examination

The following are the areas in which implementations most commonly diverge, and are where the review will concentrate. They are not a checklist to be worked through: three of them handled well counts for more than all of them handled poorly.

#	Point of examination
1	The API returns invite-only events to people who were not invited
2	Authorisation checked in the component but not in the route handler
3	The client supplies its own <code>userId</code> and the server accepts it
4	Capacity counting <code>pending</code> or <code>cancelled</code> registrations
5	Registration left open on a past, draft or cancelled event
6	Switching an event from <code>invite</code> to <code>open</code> — the fate of the invite list and of existing attendees
7	Deleting an event that carries registrations
8	A host approving a request that would exceed capacity
9	A member's registration state after withdrawing and registering again
10	Dates formatted differently on server and client, producing hydration warnings

8 Definition of done

- `npm run typecheck` and `npm run lint` complete without findings.
- `npm run build` succeeds.
- No use of `any`, no unused exports, no commented-out code.
- Styling is by CSS Modules using tokens. No inline colours and no additions to global CSS.
- Any genuinely necessary new component sits alongside the existing ones and matches them.
- The application behaves correctly as all five personas, not only as the one used during development.

9 Pull request

Pull requests are to be brief. Every one of them will be read in full.

```
## What I built
Which milestones, and the state each is in.

## Decisions
The two or three calls made that another participant would have made
differently. Where creating an event lives, and why. Where the permission
logic went, and why.

## What I did not do
An honest list. "M5 not started" is an acceptable line in a pull request.

## How to check it
The three actions a reviewer should take to see it working, including
which personas to switch between.
```

10 Applying the Claude Code loop

The loop presented in the session, applied to this specification.

Brief

Do not paste this document in whole and expect a result. Give Claude the objective and the constraints for a *single milestone*, direct it to `lib/types.ts` and `app/api/session/route.ts` as the shape to follow, and require a plan before any code is written. A wrong plan costs thirty seconds to correct; wrong code costs ten minutes.

Session

Work in slices that can be verified. Build M1, inspect it in the browser, switch persona, and only then move on. A prompt containing "and also" for the fourth time is a prompt that should have been two.

Pull request

Have Claude draft the description from the actual diff, then correct what it inferred wrongly about intent. It knows what changed; only the author knows why.

Review

Before opening the pull request, request a review against section 5 and section 7 of this document. Specificity is what makes the answer useful: "review this" returns nitpicks, whereas "check that every route handler enforces the visibility table in section 5.2" finds the defect.

Memory

When Claude is corrected on something that will recur — the API house style, where permission logic belongs, the use of CSS Modules rather than Tailwind — record it in `CLAUDE.md` so it need not be said a third time.

11 Assessment criteria

Implementations are compared on the following, in approximately this order of weight.

#	Criterion	What is looked for
1	Correctness of the rules	Whether the board leaks, and whether the API is safe when called on its own
2	Placement of logic	Whether one place answers "can this person see this event", or the question is re-answered across six files
3	Reuse	Whether the supplied kit was built upon rather than reinvented
4	The detail screen	How well a single screen serves both an attendee and a host
5	Use of Claude Code	Evident from the commit history and the pull request more clearly than participants tend to expect
6	Accuracy	Whether the pull request matches what the code actually does

REFERENCE MATERIALS

The machine-readable form of this brief is held in the repository as `TASKS.md`, and the house style it refers to as `CLAUDE.md`. Where this document and the repository disagree, the repository is authoritative.